



Neumann János Egyetem
Műszaki és Informatikai
Kar

Digitális Technika II.

L-K-GINFBAL-DIGTECH2-1

Ventilátorvezérlő rendszer PWM szabályozással

Hajda Tamás
IHXD6U

2025/26/2

Tartalom

A PROJEKTRŐL	3
FELHASZNÁLT ALKATRÉSZEK	4
MŰKÖDÉSI ELV	5
KAPCSOLÁSI RAJZ.....	7
TESZTELÉS	8
ÖSSZEGZÉS.....	10

A projektről

A projekt célja egy Arduino-alapú automatikus ventilátorvezérlő rendszer elkészítése. A rendszer egy hőmérséklet-érzékelő segítségével méri a környezeti hőmérsékletet, majd a mért érték alapján PWM-jellel szabályozza egy ventilátor vagy DC motor fordulatszámát.

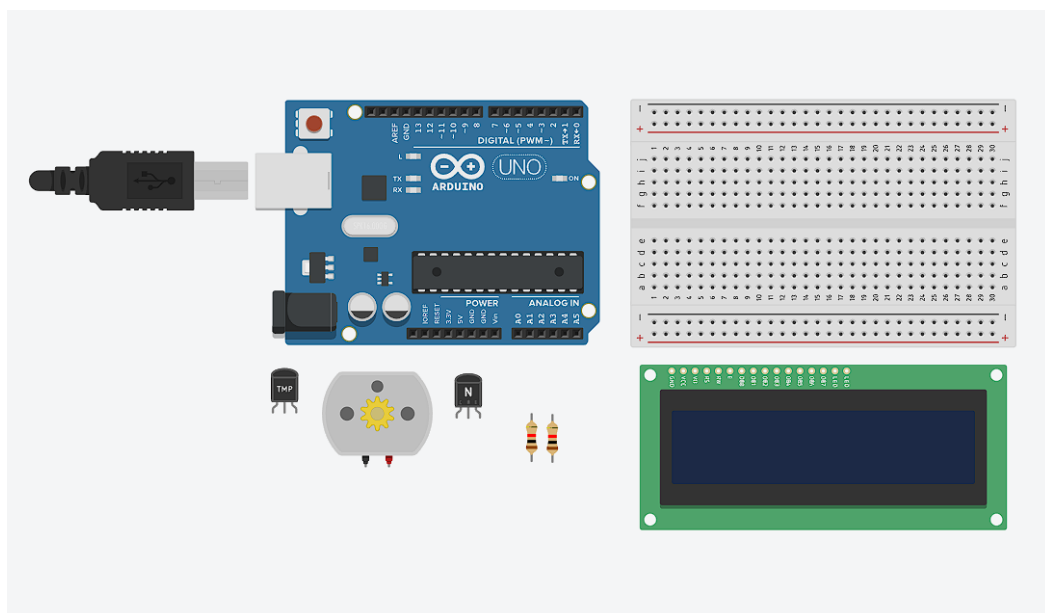
A rendszer előnye, hogy a ventilátor nem folyamatosan maximális fordulatszámon működik, hanem a hőmérséklet függvényében automatikusan változtatja a sebességét. A mért hőmérséklet és a ventilátor állapota az LCD kijelzőn is megjelenik.

Github elérés: <https://github.com/hajda08/digit2projekt>

TinkerCAD elérés: <https://www.tinkercad.com/things/kZq1GNyKOj1-pwm-venti?sharecode=pQHyw9uvfjfg1j7kT5OPngbwG533RRvCxK8NYwuGBAs>

Felhasznált alkatrészek

Alkatrész	Mennyiség	Feladat
<i>Arduino Uno R3</i>	<i>1 db</i>	<i>A rendszer központi vezérlője.</i>
<i>Próbapanel</i>	<i>1 db</i>	<i>Az áramkör összeállítása.</i>
<i>TMP36 hőmérséklet-érzékelő</i>	<i>1 db</i>	<i>Hőmérséklet mérése.</i>
<i>DC motor / ventilátor</i>	<i>1 db</i>	<i>Hűtés, légáramlás biztosítása.</i>
<i>NPN tranzisztor</i>	<i>1 db</i>	<i>A motor kapcsolása és vezérlése.</i>
<i>16x2 LCD kijelző</i>	<i>1 db</i>	<i>A hőmérséklet és a PWM érték megjelenítése.</i>
<i>Ellenállás</i>	<i>2 db</i>	<i>A tranzisztor bázisának védelmére (1k Ω és az LCD háttérvilágítás áramkorlátozására (220 Ω).</i>
<i>Vezetékek</i>	<i>szükség szerint</i>	<i>Az alkatrészek összekötése.</i>



1. ábra: A ventilátorvezérlő rendszerhez használt alkatrészek

Működési elv

Ez a ventilátorvezérlő rendszer úgy működik, hogy az Arduino Uno folyamatosan méri a környezeti hőmérsékletet a TMP36 hőmérséklet-érzékelő segítségével, ami analóg feszültségjelet ad ki. Ez arányos változik a hőmérséklet változásával.

A beolvasott értéket először feszültséggé alakítja, majd ebből számítja ki a hőmérsékletet Celsius-fokban. Figyelembe kell venni, hogy 0 °C körülbelül 0,5 V kimeneti feszültség van, ezért ezt le kell vonni a hőmérséklet értékének számításakor.

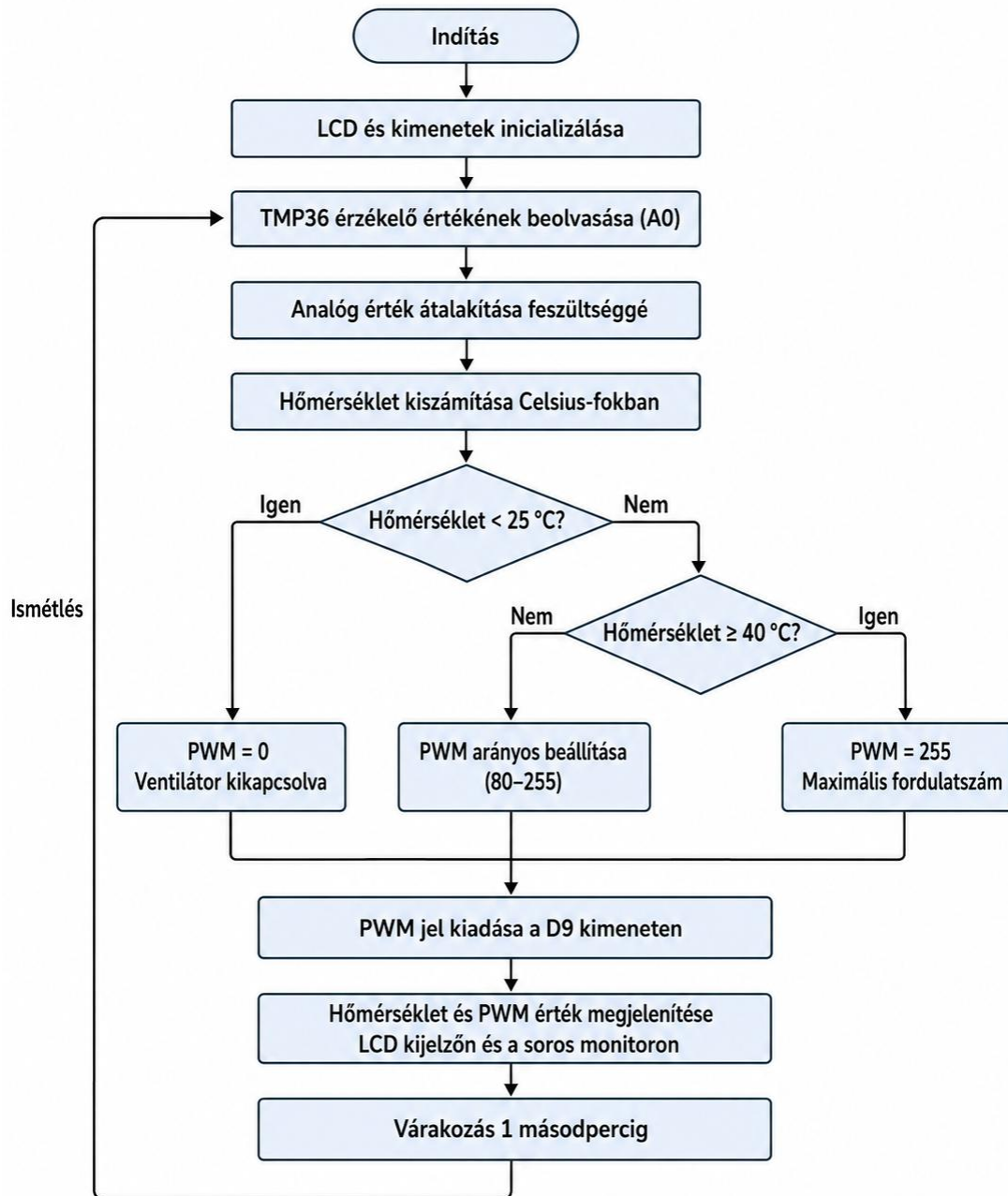
Ezután a program a kiszámított hőmérséklet alapján meghatározza a ventilátor fordulatszámát, aminek a vezérlése PWM jellel történik. A PWM, vagyis impulzusszélesség-moduláció lényege, hogy az Arduino nagyon gyorsan kapcsolgatja a kimeneti jelet ki és be. Ezeknek az aránya határozza meg, hogy a motor mekkora teljesítményt kap.

Alacsony (jelen esetben 25 °C alatt) hőmérséklet esetén kikapcsolva marad. Ha a hőmérséklet emelkedik (25 °C – 40 °C), akkor a központi vezérlő fokozatosan növeli a PWM értékét, ezáltal a ventilátor is gyorsabban fog forogni. Magas hőmérséklet (+ 40 °C) esetén a ventilátor maximális fordulatszámon működik.

A ventilátor nem közvetlenül van az Arduinora kötve, hanem egy NPN tranzisztoron keresztül működik. Azért szükséges ez a megoldás, mert a motor nagyobb áramot igényelhet, mint amit a központi vezérlő biztonságosan biztosítani tud. Ezért csak a tranzisztort vezérli, az pedig kapcsolja a motor áramkörét.

Működés közben az aktuális hőmérsékletet és a ventilátor állapotát a 16x2 LCD kijelző jeleníti meg. Így könnyen nyomon tudjuk követni, hogy mekkora a mért hőmérséklet és milyen állapotban működik a ventilátor.

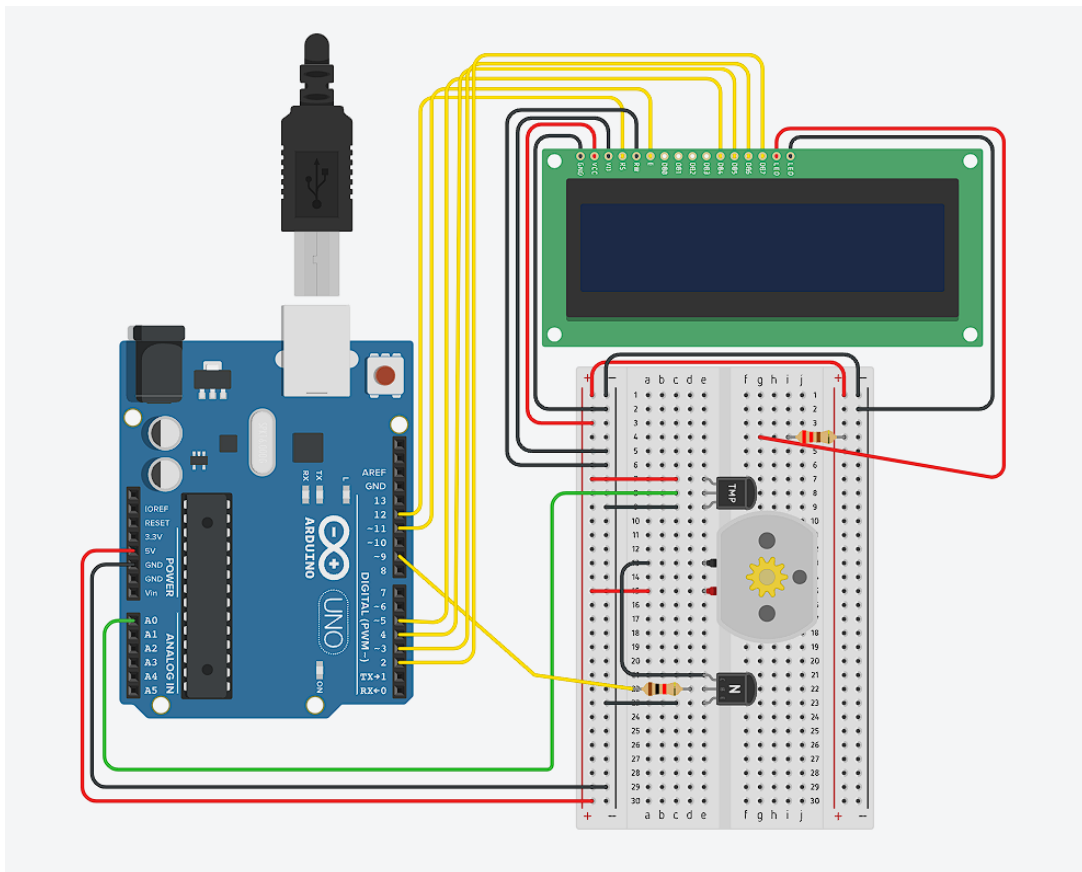
A ventilátorvezérlő rendszer folyamatábrája



2. ábra: A ventilátorvezérlő rendszer működésének folyamatábrája

Kapcsolási rajz

Az alábbi ábra a ventilátorvezérlő rendszer gyakorlati bekötését mutatja be. A kapcsolásban látható az Arduino Uno, a TMP36 hőmérséklet-érzékelő, a 16x2 LCD kijelző, valamint a ventilátorként szolgáló DC motor tranzisztoros vezérlése.



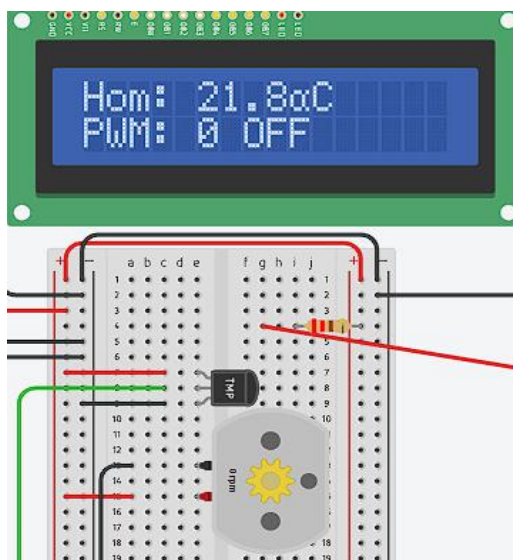
3. ábra: A ventilátorvezérlő rendszer kapcsolási rajza

Ennek a célja, hogy szemléltessen minden fontos összeköttetést, amely a rendszer működéséhez szükséges.

Tesztelés

1. eset: hőmérséklet $<25\text{ }^{\circ}\text{C}$

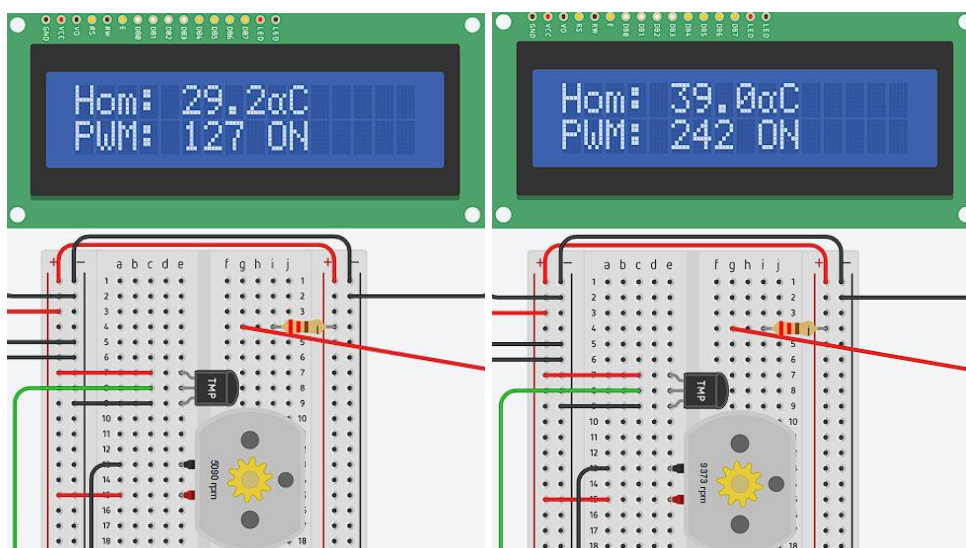
A mért hőmérséklet alacsonynak számít, ezért nincs szükség hűtésre. Ilyenkor a PWM értéke 0.



4. ábra: Alacsony hőmérséklet esetén a ventilátor kikapcsolt állapotban van

2. eset: hőmérséklet $25\text{--}40\text{ }^{\circ}\text{C}$

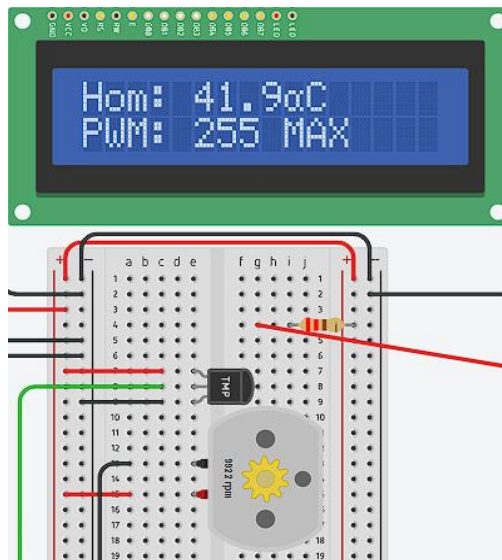
Ebben a tartományban a rendszer már szabályozza a ventilátor fordulatszámát. A hőmérséklet emelkedésével arányosan növeli a PWM értéket. Minél magasabb a hőmérséklet, annál gyorsabban forog a ventilátor és növekszik a PWM.



5-6. ábra: PWM és a ventilátor fordulatszámának növekedése

3. eset: hőmérséklet $> 40^{\circ}\text{C}$

Ha a hőmérséklet meghaladja a 40°C -ot, akkor a rendszer maximális hűtést biztosít. Ebben az esetben a program a legnagyobb PWM értéket állítja be.



7. ábra: Magas hőmérsékletnél maximális értéket vesz fel a PWM.

A tesztelés alapján a rendszer a hőmérséklet változására megfelelően reagál. Alacsony hőmérsékleten kikapcsolva marad, $25\text{--}40^{\circ}\text{C}$ tartományban fokozatosan növeli a fordulatszámot, magas hőmérsékleten pedig maximális teljesítményt nyújt.

Összegzés

A projekt során egy Arduino-alapú ventilátorvezérlő rendszer került megvalósításra, ami TMP36 hőmérséklet-érzékelő segítségével érzékeli a környezeti hőmérsékletet, majd az Arduino a kapott érték alapján PWM jelet állít elő.

A PWM szabályozás lehetővé teszi, hogy a ventilátor ne csak ki- és bekapcsolt állapotban működjön, hanem a hőmérséklet változásának megfelelően fokozatosan növelje vagy csökkentse a fordulatszámát.

A 16x2 LCD kijelző segítségével a rendszer működése könnyen nyomon követhető. A tesztelés alapján a rendszer megfelelően reagált a különböző hőmérsékleti tartományokra, ezért a működése megfelelt az előzetes elvárásoknak.

Összességében a projekt jól szemlélteti a hőmérsékletmérés, az analóg jel feldolgozása, a PWM alapú motorvezérlés és az LCD kijelzés gyakorlati alkalmazását.